

Incident Response Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC7.3/CC7.4/CC7.5 · HIPAA §164.308(a)(6) · GDPR Art. 33 (breach notification) **Companion documents:** Access Control Policy · Encryption Policy · HIPAA SRA v1 · BAA_DRAFT.md §3 (breach reporting).

1. Purpose

Give an operator under stress a script to follow — detection, triage, containment, notification, recovery, post-mortem — not prose to parse. Every window in this document is one we can actually meet with the team of two we have today.

2. Scope

- Security incidents on RailCall-operated infrastructure: gateway (`railcall-core.onrender.com`), Render Postgres, VPS `157.230.177.45` (railcall.ai + installer mirror), the issuer signing seed, Stripe account, GitHub organisation.
- Product-level incidents affecting shipped stations: forged receipts, entitlement compromise, integrity failures.
- Third-party incidents that reach us: Render outage, Stripe outage, Anthropic outage, GitHub compromise, dependency supply-chain compromise.

OUT of scope: incidents entirely on the customer's own machine that never touch our infrastructure or invalidate a RailCall-signed artifact. Those belong to the customer's own §164.308(a)(6) program; we assist under the BAA when we're a business associate for that deployment.

3. Severity classification

Severity	Definition	Response window (acknowledge / contain / notify)	Example
SEV-1 Critical	Trust root compromised OR customer data at risk OR paid tier globally unavailable	30 min / 4 h / 10 days per BAA §3.3	Issuer seed leaked; receipts being forged in the wild; gateway serving PHI
SEV-2 High	Non-root credential compromised OR partial paid-tier failure OR audit evidence integrity compromised	2 h / 24 h / per contract	Stripe key leaked; audit chain break detected; one install seed leaked
SEV-3 Medium	Degraded service, contained, no customer impact	24 h / 5 business days / discretionary	Gateway 5xx spike; Postgres slow; one webhook delivery failing
SEV-4 Low	Observed anomaly, no operational impact	Next business day / at your leisure / no	One-off failed login attempt; noisy log; false positive

Escalate up on any of: legal/regulatory exposure discovered, customer directly harmed, media attention likely, or when in doubt. The cost of over-escalating is a meeting; the cost of under-escalating is a breach that turns into willful neglect under OCR review.

4. Detection — what triggers an incident

Sources ranked by trust:

1. **Customer report** — email to `security@railcall.ai`. Treat every report as at least SEV-3 until triaged.
2. **Automated invariants failing** — see §4.1 for the specific ones.
3. **Third-party notification** — Render status page, Stripe security alert, GitHub security alert on a dependency, upstream CVE.
4. **Our own observability** — Render logs, `pm2 logs railcall-website`, `docker logs probo-probo-1`, ad-hoc `curl` health checks.
5. **Internal review** — Sami/Pat noticing something odd during normal work.

4.1 Automated invariants (checked on every relevant read)

Invariant	Where	On break
Receipt signature verifies against pinned pubkey	<code>entitlement.py</code> <code>verify_against_install</code>	Receipt returns <code>SIG_FAIL</code> (not <code>SIG_UNSIGNED</code> — that's honest legacy). Any <code>SIG_FAIL</code> in a customer report → SEV-1.
Audit chain <code>prev_hash</code> consecutive	<code>audit_chain.py</code> <code>verify()</code>	Returns <code>chain_intact=False</code> + locates the break. Customer or auditor running <code>verify</code> sees this. SEV-2 by default; SEV-1 if the break correlates with a known access event.
Issuer pubkey served matches station pin	<code>curl /v1/issuer/pubkey</code> returns the pinned hex	Mismatch = SEV-1 (either the seed was rotated without a station release, or the seed was replaced without our knowledge).
<code>RAILCALL_ISSUER_SEED</code> set on gateway	<code>/v1/entitlement/mint</code> returns 200 for a valid paid caller	503 = billing broken; SEV-2 until the seed is restored.
PHI redaction still enforced	<code>phi_guard</code> still importable at station boot	ImportError in station logs = SEV-2 for any install with <code>RAILCALL_PHI_MODE=on</code> ; SEV-3 otherwise.
Seat cap enforced	<code>/v1/seat/checkin</code> refuses over-capacity installs with 402	If a customer reports "I paid for 3 but 5 machines work", check <code>/v1/seat/status</code> — if it agrees, treat as SEV-2 (billing integrity).

5. Roles

Two-person team. Roles rotate by availability, not by title.

- **Incident Commander (IC):** whoever picks up first — owns the incident until handoff or closure. Their job is decisions and communication, NOT to be the one debugging (that's the technical responder). In a two-person team, one person is often both, but the ROLES are distinct so nothing gets dropped when one person is on a plane.
- **Technical Responder (TR):** the one making changes. Rotates a screen with the IC on a Signal call for anything SEV-2 or worse.

- **Security Officer (SO):** Sami. Final decision authority on customer notification, regulatory reporting, and public disclosure. Does not need to be the IC.

6. Runbook — the actual steps

6.1 Every incident (any severity)

1. **Open a shared note.** `~/incidents/YYYY-MM-DDThhmm-<slug>.md`. Time-stamped, append-only, on Sami's box. This IS the incident record; do not rely on memory or chat scrollback.
2. **State the invariant that broke.** Not the symptom — the invariant. "The receipt signed by X does not verify against the pubkey pinned in station-v0.16" is an invariant. "A customer emailed us angry" is a symptom.
3. **Assign IC + TR.** In writing, in the note.
4. **Set severity from §3.** Update at any time; log every change with the reason.
5. **Take one containment action within the response window.** Even a bad containment (e.g., "took gateway offline for 5 min") is better than paralysis. Log it.
6. **Restore.** Follow §7 per-scenario runbooks.
7. **Notify per §8.**
8. **Post-mortem within 5 business days.** §9.

6.2 Notifications inside the team

- SEV-1: Signal call within 30 min. If IC can't reach TR/SO, escalate to phone.
- SEV-2: Signal message within 2 h + Signal call for anything not obvious.
- SEV-3: Signal message within 24 h.
- SEV-4: Log in the note; discuss at the next sync.

7. Scenario runbooks

Each below is a decision tree — no "figure it out" cells.

7.1 Issuer signing seed compromise or loss

Severity: SEV-1 unconditionally.

1. **Assess reach.** Has ANY receipt or entitlement been minted against this seed since compromise-suspected? If yes: every one is untrusted. If no: recovery is clean.

2. **Rotate immediately.** Do NOT wait to schedule downtime. - Run `python3 tools/mint_issuer_keypair.py mint --seed-out ~/railcall-issuer-seed-YYYYMMDD.txt` on Sami's box. - Back up the new seed to 1Password + offline paper BEFORE proceeding.
3. **Deploy the rotation.** Follow the Encryption Policy §7 issuer-seed-lost runbook (edit `entitlement.py` pin, cut station-vN+1, re-pin `install.sh / install.ps1`, deploy contrib mirror, `ssh sami@157.230.177.45 && cd ~/railcall-contrib && git pull && pm2 restart railcall-website`).
4. **Set new seed on Render.** Service-level env var `RAILCALL_ISSUER_SEED`. Verify: `curl https://railcall-core.onrender.com/v1/issuer/pubkey` returns the new pubkey.
5. **Communicate.** Email every paying customer: "we rotated the issuer key. Reinstall your station via `curl -fsSL https://railcall.ai/install.sh | bash`, then `railcall activate` to re-mint your entitlement." Include the reason at your discretion — if compromised, say so; the trust cost of hiding it is higher than the trust cost of saying it.

7.2 Individual install signing seed compromise

Severity: SEV-2 (single customer scope).

1. Confirm reach — is this install signing anything the customer or a downstream verifier trusts? Usually yes (their `railcall demo` receipts).
2. Instruct the customer to run `railcall rotate-key`. This mints a fresh keypair, archives the old public doc, so pre-rotation receipts stay verifiable via `verify --key <archived>`.
3. Optionally: reinstall the station cleanly to be sure nothing else is compromised.
4. If unclear whether the compromise is limited to the seed — treat as SEV-1 for the customer's own workflow data (not for RailCall as a whole).

7.3 Audit chain break

Severity: SEV-2 default; SEV-1 if correlated with a known access event.

1. Read `verify()` output — it locates the break to a specific record index.
2. Before that index, the chain is intact; after that index, it's rewritten.
3. If the customer holds their own audit logs (usual case), the customer is the SoT — we assist their investigation but do not have their evidence.
4. If the break is in a hosted-service log (`Render` logs the gateway keeps), our copy is the SoT — extract the pre-break slice, treat post-break as tampered.
5. Document the break, its context, and the recovery in the incident note. `verify()` returns the caveat text — copy it in verbatim so nobody accidentally over-claims.

7.4 Gateway outage (Render)

Severity: SEV-3 usually; SEV-2 if it lasts > 1 hour or during a paid customer's mint window.

1. Check Render status: <https://status.render.com>.
2. Free-tier installs are unaffected (they do not depend on the gateway). Communicate this on Twitter/Discord if outage is public.
3. Paid installs cannot mint NEW entitlements but existing entitlements verify locally until expiry — this is by design, not a bug.
4. If Render is up and only our service is down: check `docker logs` via the Render dashboard, roll back to the previous deploy if the current one is broken.
5. Post-outage: if any paid customer's entitlement expired during the outage, offer them an extension via email.

7.5 Stripe outage or credential compromise

Severity: SEV-2.

1. **Outage:** existing subscribers keep being charged by Stripe independently of our service. New checkout is blocked; add a banner to the pricing page pointing to Discord for updates.
2. **Credential compromise:** rotate ALL Stripe keys immediately (Standard + every Restricted). Update `STRIPE_SECRET_KEY` + `STRIPE_WEBHOOK_SECRET` on Render. Verify webhook signature verification is still passing (`POST /v1/webhooks/stripe` with unsigned body returns 400 "Unable to extract timestamp and signatures").
3. Search Stripe dashboard for any suspicious sessions/customers created during the compromise window. Refund fraudulent charges.

7.6 VPS `157.230.177.45` (railcall.ai) compromise

Severity: SEV-2.

1. `ssh sami@157.230.177.45` — check `who`, `last`, `ss -tulpn`, `pm2 logs`.
2. Rotate the SSH key (`~/.ssh/id_ed255199`) — add a new key to `authorized_keys`, remove the old, verify you can still log in with the new before disconnecting.
3. If installer or station tarball on the mirror was tampered with — `railcall-contrib/website-v2/public/install.sh` or `.tar.gz` — that's SEV-1: users installing from the mirror got a compromised binary. Byte-compare against `railcall-core/install.sh` and the GitHub release tarball. If mismatch: fix the mirror, redeploy, and email every user who installed during the compromise window telling them to reinstall.

7.7 Dependency / supply chain compromise

Severity: SEV-2 default; SEV-1 if the compromised package is on the station-signing or gateway-secret path.

1. Identify which package + version. Search `railcall-core/requirements.txt` and `railcall-engine`'s Python imports.
2. If on the signing path (`cryptography` package etc.) — treat as SEV-1: might have signed with a compromised signer. Rotate signing seeds.
3. Pin the fixed version; re-run every test suite; re-cut station release if any station-bundled dependency is affected.
4. Retrospectively verify: are any of our receipts in customers' hands signed against the compromised version? If yes, communicate.

8. External notification

- **Customers affected:** direct email from `security@railcall.ai` within the response window in §3. Template lives in `compliance/templates/breach_notice.md` (TO BE WRITTEN). Include: what happened, when, what data was affected, what we've done, what they should do, when we'll next update them.
- **Covered entities with a BAA:** per BAA §3.3, within 10 days of discovery of a breach affecting their PHI. Draft template in `legal/BAA_DRAFT.md` §3 — DO NOT SEND without counsel review the first time.
- **Regulators:**
- **HHS OCR** for HIPAA breaches affecting 500+ individuals: 60 days per § 164.408. Fewer than 500: annual summary.
- **State AGs:** varies by state; assume 30 days unless research proves otherwise for the specific state.
- **GDPR supervisory authority:** 72 hours per Art. 33 if RailCall is a processor for EU personal data.
- **Public disclosure:** SO decides. Default: proactive if any customer is affected, silent if the incident is fully contained pre-impact and disclosure serves no protective purpose. Do NOT lie by omission — if asked, answer honestly.

9. Post-incident review

Within 5 business days of closure for SEV-1/SEV-2. Simple format:

1. **What happened** (timeline in the incident note, cleaned up).

2. **Why it happened** — root cause, not proximate cause. "The pod restarted" is proximate; "we had no memory limit set, so a leak in dep X ate the pod" is root.
3. **What we did well.**
4. **What we did badly.** Blameless — the process is what failed, not the person.
5. **Action items** with owners and due dates. Each one is a real ticket, not a wishlist.

Post-mortems land in `~/incidents/post-mortems/` (Sami's box) and are shared with any customer we notified about the incident. Redact only what genuinely can't be shared (customer identity, active security-through-obscurity mitigations); everything else is public to the affected parties.

10. Testing

Once a quarter, at least one scenario from §7 is tabletop-exercised — a 30 min Signal call where IC + TR walk through the runbook against a hypothetical trigger. Log the exercise as a "SEV-4 (tabletop)" incident so the process itself gets its reps.

At least once per year, one scenario is executed end-to-end in a staging environment (or the real environment during a known low-traffic window) — issuer seed rotation is the natural annual exercise since it's the highest-blast-radius runbook.

11. Related documents

- **Access Control Policy** — who has the credentials that get compromised in §7.1, §7.5, §7.6.
- **Encryption Policy** — the algorithms and key lifecycles referenced from §7.1, §7.2, §7.7.
- **HIPAA SRA v1** — the threat catalog (§3) whose realizations this policy responds to.
- **BAA_DRAFT.md** — §3.3 breach reporting window.
- **audit_chain.py + test_audit_chain.py** — the primary detection tooling for §4.1 invariant "audit chain intact".